Dataflow Analysis

Principles of Program Analysis

Table of Contents

Running Example — Factorial Skeleton

Throughout these slides we use the following **labelled WHILE program**:

```
[x := a]^1 ;
[y := 1]^2 ;
while [x > 1]^3 do
  [y := x * y]^4 ;
  [x := x - 1]^5
[z := y]^6 ;
[w := y]^7
```

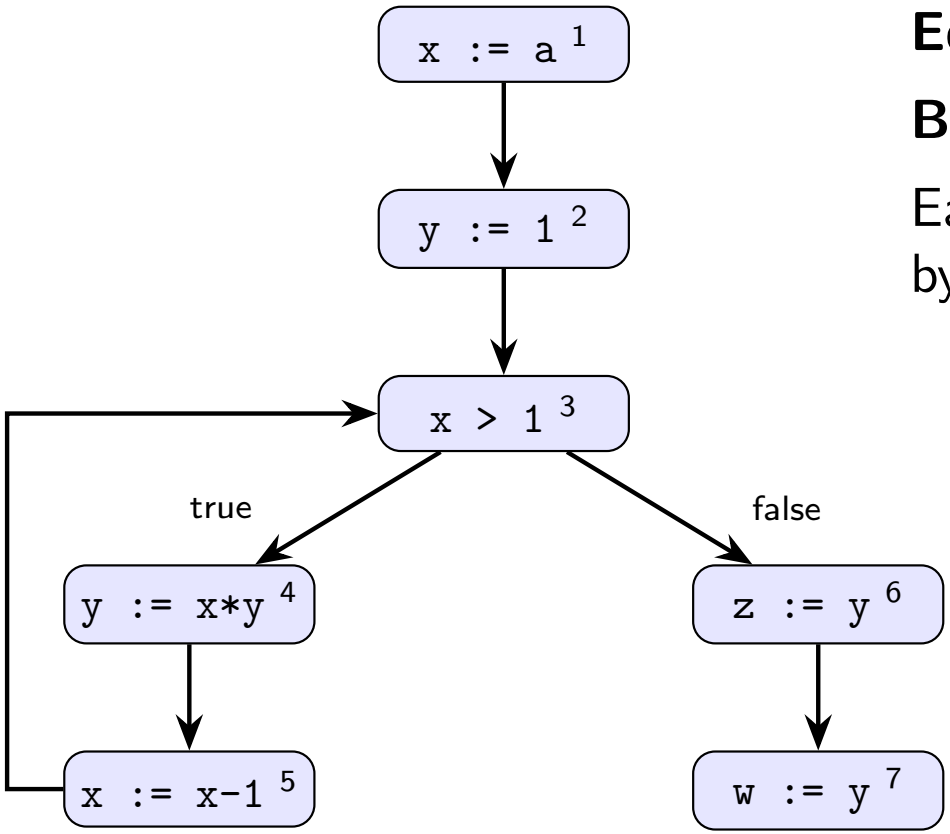
Superscripts = labels

Each statement / condition carries a unique label. Labels are the unit of analysis.

Running Example — Discussion

- The program computes the **factorial** of a into y by multiplying $y \leftarrow y \cdot x$ and decrementing x each iteration.
- Each statement carries a **unique label** so that the analysis can refer to a specific program point without ambiguity. The same variable can behave differently at label 3 (loop test) vs. label 5 (update).
- Labels 6 and 7 are added deliberately: $[z := y]^6$ is a *dead assignment* because z is never used again — $[w := y]^7$ uses only y. We will verify this formally using Live Variables Analysis.

Control Flow Graph (CFG)



Nodes = labelled statements.

Edges = possible control transfers.

Back-edge $5 \rightarrow 3$ represents the loop.

Each time around the loop, x decreases by 1 and y accumulates the product.

Control Flow Graph — Discussion

- The CFG is the **foundation** for all dataflow analyses.
- An edge $u \to v$ means execution *may* flow from u to v (it is a *may* graph, not *must*).
- The **back-edge** $5 \to 3$ is what makes loop analysis hard: information from *later* in the program (label 5) can affect *earlier* points (label 3), requiring iterative fixed-point computation.
- Label 3 has **two predecessors** (label 2 and label 5) and two successors (label 4 and label 6) — it is a *join point* and a *branch point* simultaneously.

Use-Definition (UD) Chains

Definition

For each **use** of a variable v at label ℓ, the UD chain lists all definitions of v that can *reach* that use without being killed.

Use site	**Var**	**Reaching definitions**
$[x > 1]^3$	x	$\{[x := a]^1, [x := x-1]^5\}$
$[y := x * y]^4$	x	$\{[x := a]^1, [x := x-1]^5\}$
$[y := x * y]^4$	y	$\{[y := 1]^2, [y := x * y]^4\}$
$[z := y]^6$	y	$\{[y := 1]^2, [y := x * y]^4\}$
$[w := y]^7$	y	$\{[y := 1]^2, [y := x * y]^4\}$

Use-Definition Chains — Discussion

- A UD chain answers: **"where could this value have come from?"**
- At label 3, x may have been set by the initialisation ($[x := a]^1$) *or* by the loop decrement ($[x := x-1]^5$) — both survive to reach the test.
- At label 4, y may come from the initialisation ($[y := 1]^2$) or from the previous loop iteration ($[y := x * y]^4$ feeds back to itself). This self-loop in the DU chain signals recursively defined data.
- Compilers use UD chains for **constant folding** (if the UD chain has a single entry with a constant value, the use can be replaced by that constant) and **register allocation** (keep a variable in a register as long as it has pending uses).

Definition-Use (DU) Chains

Definition

For each **definition** of v at label ℓ, the DU chain lists all uses of v that this definition can reach.

Definition	**Var**	**Reachable uses**
$[x := a]^1$	x	$\{[x > 1]^3, [y := x * y]^4\}$
$[y := 1]^2$	y	$\{[y := x * y]^4, [z := y]^6, [w := y]^7\}$
$[y := x * y]^4$	y	$\{[y := x * y]^4, [z := y]^6, [w := y]^7\}$
$[x := x-1]^5$	x	$\{[x > 1]^3, [y := x * y]^4\}$

Observation

$[x := a]^1$ and $[x := x-1]^5$ both reach the loop condition — x is overwritten inside the loop.

Definition-Use Chains — Discussion

- DU chains answer: **"where does this value get consumed?"**
- A definition with an **empty DU chain** is never used — it is dead code and can be removed.
- Notice that z has *no DU entry* in the table above: $[z := y]^6$ defines z, but z is never in any use site. This is the dead assignment we will confirm via Live Variables Analysis.
- UD and DU chains together form the **def-use web**, which is the core data structure behind SSA (Static Single Assignment) form used by modern compilers (LLVM, GCC).

Live Variables — Intuition

Definition

A variable v is **live** at program point p if there exists a path from p to a *use* of v that does *not* pass through a redefinition of v.

- **Direction:** Backward — information flows from uses toward definitions
- **Lattice:** $(\mathcal{P}(Var), \subseteq)$; join $= \cup$
- **Use:** Dead-code elimination, register allocation

Example

$[w := y]^7$ uses y but *not* z, so z has no use after label 6. Therefore z is *never live* at any point $\Rightarrow$ $[z := y]^6$ is **dead code** and can be eliminated.

Live Variables — Discussion

- A variable is live if its **current value matters** for future execution. Storing a dead value wastes a register or memory slot.
- The analysis runs **backward** because liveness propagates from the point of *use* back toward the point of *definition* — the opposite direction to program execution.
- At a **join point** (e.g. label 3), a variable is live if it is live on *any* successor path — hence the join is *union*. This makes the analysis a safe over-approximation (a variable may be considered live on a path that is never actually taken).
- Live variables analysis is used by compilers for **register allocation**: only live variables need to occupy registers.

Live Variables — Transfer Functions

For each label ℓ, define:

$$f_\ell(X) = \underbrace{gen[\ell]}_{\text{vars used at } \ell} \cup (X \setminus \underbrace{kill[\ell]}_{\text{var defined at } \ell})$$

Label	Statement	gen	kill
1	$x := a$	$\{a\}$	$\{x\}$
2	$y := 1$	$\emptyset$	$\{y\}$
3	$x > 1$	$\{x\}$	$\emptyset$
4	$y := x * y$	$\{x, y\}$	$\{y\}$
5	$x := x - 1$	$\{x\}$	$\{x\}$
6	$z := y$	$\{y\}$	$\{z\}$
7	$w := y$	$\{y\}$	$\{w\}$

Live Variables — Transfer Functions (Discussion)

- **gen** = variables whose values are *read* at this label. They must already be live *before* the label executes.
- **kill** = the variable *written* at this label. Any earlier liveness of that variable ends here (the new definition makes the old value irrelevant).
- At label 5 ($x := x - 1$), x appears in *both* gen and kill: the right-hand side *reads* the old x (gen), and the left-hand side *overwrites* x (kill). The order matters: gen is applied *before* kill in the transfer function.
- Labels 6 and 7 both have $gen = \{y\}$, confirming y is live throughout the exit sequence.

Live Variables — Fixed-Point Solution

Backward equations: $LV_{\text{entry}}[\ell] = f_\ell(LV_{\text{exit}}[\ell]), \quad LV_{\text{exit}}[\ell] = \bigcup_{\ell' \in succ(\ell)} LV_{\text{entry}}[\ell']$

Point	$\mathbf{LV_{entry}}$	$\mathbf{LV_{exit}}$
1	$\{a\}$	$\{x, a\}$
2	$\{x\}$	$\{x\}$
3	$\{x, y\}$	$\{x, y\}$
4	$\{x, y\}$	$\{x, y\}$
5	$\{x, y\}$	$\{x, y\}$
6	$\{y\}$	$\{y\}$
7	$\{y\}$	$\emptyset$

Note

z never appears in any LV_{entry} set $\Rightarrow$ $[z := y]^6$ is **dead code** (label 7 uses only y, not z).

Live Variables — Fixed-Point (Discussion)

- The equations are solved **iteratively**, starting from $\emptyset$ at all points, and applying the backward transfer functions until no set changes.
- The **back-edge** $5 \to 3$ forces $\{x, y\}$ to propagate into the loop body: both x and y are needed for the next iteration of the loop.
- At label 1, $LV_{\text{exit}} = \{x, a\}$ because: after assigning $x := a$, both x (needed by the loop) and a (used as the right-hand side) are live just before label 1.
- z and w appear in no LV_{entry} set: z is dead (no subsequent use); w is only assigned, never read after label 7.

Monotone Frameworks

Definition

A **Monotone Framework** is a pair (L, F) where:

1. L is a **complete lattice** $(L, \sqsubseteq, \sqcup, \sqcap, \top, \bot)$
2. F is a set of **monotone** transfer functions $f : L \to L$:
   $x \sqsubseteq y \implies f(x) \sqsubseteq f(y)$
3. F is closed under composition and contains the identity.

Live Variables as a Monotone Framework

- $L = (\mathcal{P}(Var), \supseteq)$ (more vars = more info)
- $f_\ell(X) = gen[\ell] \cup (X \setminus kill[\ell])$
- Monotone: $X \subseteq Y \Rightarrow X \setminus k \subseteq Y \setminus k \Rightarrow f(X) \subseteq f(Y)$ ✓

Monotone Frameworks — Discussion

- **Monotonicity guarantees termination**: because the lattice has finite height and transfer functions only move upward under $\sqsubseteq$, the iterative fixed-point computation converges in finitely many steps. Without it, the worklist algorithm might loop forever.
- A **complete lattice** ensures that any subset of elements has a least upper bound (join $\sqcup$) and a greatest lower bound (meet $\sqcap$). For Live Variables, the lattice is $(\mathcal{P}(\mathit{Var}), \supseteq)$: *more* variables in the set means *more* information (safer approximation).
- The identity function is required in F because a "no-op" statement (e.g. a label with no kill or gen) should leave the analysis state unchanged.

Distributive Frameworks

Definition

A monotone framework is **distributive** if every $f \in F$ satisfies:

$$f(\ell_1 \sqcup \ell_2) = f(\ell_1) \sqcup f(\ell_2) \quad \forall\, \ell_1, \ell_2 \in L$$

Live Variables is Distributive

$$\begin{aligned} f_\ell(X \cup Y) &= \mathit{gen} \cup \big((X \cup Y) \setminus \mathit{kill}\big) \\ &= \mathit{gen} \cup (X \setminus \mathit{kill}) \cup (Y \setminus \mathit{kill}) = f_\ell(X) \cup f_\ell(Y) \quad \checkmark \end{aligned}$$

Classical Distributive Analyses

Reaching Definitions	Available Expressions
Live Variables	Very Busy Expressions

Distributive Frameworks — Discussion

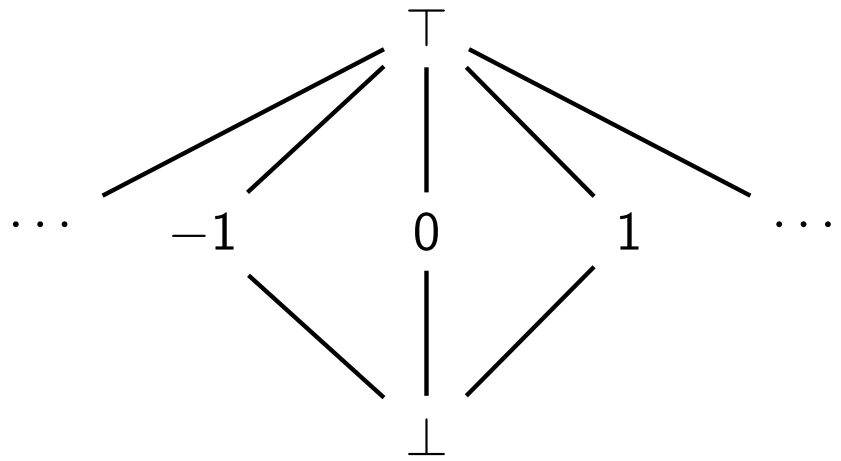
- Distributivity means the transfer function **does not lose information when inputs are merged**. It is a strictly stronger condition than monotonicity.
- Intuitively: it does not matter whether you merge two incoming analysis states *before* applying f or *after* applying f — the result is the same.
- All four classical gen/kill analyses are distributive because set union distributes over set difference: $(X \cup Y) \setminus k = (X \setminus k) \cup (Y \setminus k)$.
- Distributivity implies that the **fixed-point solution** (defined later as MFP) equals the **ideal path-based solution** (defined later as MOP), meaning the worklist algorithm gives the *theoretically best* result.

Constant Propagation — Lattice

Goal

Determine which variables hold *constant* values at each program point.

Flat lattice per variable (constants are *incomparable*):



- $\bot$ = unreachable code path
- $c \in \mathbb{Z}$ = variable definitely holds constant c
- $\top$ = non-constant / unknown

$\bot \sqsubseteq c \sqsubseteq \top$

$c_1 \not\sqsubseteq c_2$ for $c_1 \neq c_2$

Transfer function for $[y := e]^\ell$

Constant Propagation — Lattice (Discussion)

- The lattice is **flat**: $\bot$ at the bottom, all integer constants at the same intermediate level, and $\top$ at the top. Two *different* constants join to $\top$ (we cannot choose between them).
- $\bot$ represents *unreachable* code: if a variable is $\bot$, the program can never reach that point — a stronger claim than any constant.
- If either operand of an expression is $\top$, the result is $\top$ (we cannot know the value at compile time).
- The full lattice maps *all program variables* to their per-variable values. The height of this product lattice is 3 (for each variable), ensuring termination of the fixed-point iteration.

Constant Propagation — NOT Distributive

Counterexample: $[y := x * x]^\ell$

Two incoming states: $\sigma_1 = \{x \mapsto 1\}, \quad \sigma_2 = \{x \mapsto -1\}$

Apply f per path, then join:

$$f_\ell(\sigma_1) \sqcup f_\ell(\sigma_2) = \{y \mapsto 1\} \sqcup \{y \mapsto 1\} = \{y \mapsto 1\}$$

Join paths first, then apply f:

$$f_\ell(\sigma_1 \sqcup \sigma_2) = f_\ell(\{x \mapsto \top\}) = \{y \mapsto \top\}$$

$$f(\sigma_1 \sqcup \sigma_2) \neq f(\sigma_1) \sqcup f(\sigma_2) \quad \Rightarrow \quad \textbf{NOT distributive!}$$

Constant Propagation — Discussion

- Although both $x = 1$ and $x = -1$ produce $y = 1$ (since $(\pm 1)^2 = 1$), the **fixed-point (MFP) analysis** (defined in the next section) **loses this fact** by merging x to $\top$ before applying f. It conservatively reports $y = \top$.
- The **ideal path-based (MOP) solution** (defined in Section 9) would correctly deduce $y = 1$ by following each path separately — but it is undecidable in general.
- The root cause: **arithmetic can exploit algebraic structure** ($x^2 \geq 0$ for all real x; x^2 is even for even x, etc.) that the flat lattice cannot capture. Distributive analyses only use set-theoretic structure, which is preserved.
- SCCP (Sparse Conditional Constant Propagation) works around this by combining constant propagation with reachability analysis, giving better results in practice while remaining computable.

MFP Solution — Worklist Algorithm

Maximum Fixed Point (MFP)

The **least** solution (w.r.t. $\sqsubseteq$): most informative safe approximation.

Let flow = CFG edges for **forward**;
flow = reversed edges for **backward**.

ℓ_{bound} = entry node (fwd) or exit node (bwd).

ι = initial value at boundary ($\emptyset$ for most analyses).

1: $\text{Analysis}[\ell_{\text{bound}}] \leftarrow \iota$
2: $\text{Analysis}[\ell] \leftarrow \bot \quad \forall\, \ell \neq \ell_{\text{bound}}$
3: $W \leftarrow \text{flow}$
4: **while** $W \neq \emptyset$ **do**
5: remove $(u \rightarrow v)$ from W
6: **if** $f_u(\text{Analysis}[u]) \not\sqsubseteq \text{Analysis}[v]$ **then**
7: $\text{Analysis}[v] \leftarrow \text{Analysis}[v] \sqcup f_u(\text{Analysis}[u])$
8: $W \leftarrow W \cup \{(v \rightarrow w) \mid (v \rightarrow w) \in \text{flow}\}$
9: **end if**
10: **end while**

MFP Solution — Discussion

- The worklist avoids **reprocessing** nodes whose input has not changed, making it far more efficient than naive repeated iteration over all nodes.
- **Termination** is guaranteed by monotonicity: Analysis values can only grow ($\sqsubseteq$), and the lattice has finite height, so the loop must stop.
- The **order** in which edges are processed affects speed but not the final result — any order converges to the same fixed point. A FIFO queue (BFS order) is typical; RPO (reverse post-order) is faster for forward analyses.
- The re-insertion of outgoing edges after a change is key: it ensures that all downstream nodes see the updated information.

Worklist Trace — Reaching Definitions (1/2)

Running example. Definitions: $(x,1),(y,2),(y,4),(x,5)$. Each row: pick edge $(u \to v)$, compute $f_u(\mathrm{RD}[u])$, join into $\mathrm{RD}[v]$.

Step	**Edge**	**RD$[v]$ after join**	**Grows?**
1	$1 \to 2$	$\{(x,1)\}$	✓
2	$2 \to 3$	$\{(x,1),(y,2)\}$	✓
3	$3 \to 4$	$\{(x,1),(y,2)\}$	✓
4	$4 \to 5$	$\{(x,1),(y,4)\}$	✓

So far

Linear propagation along $1 \to 2 \to 3 \to 4 \to 5$. No back-edge processed yet — loop definitions not visible at label 3.

Worklist Trace — Reaching Definitions (2/2)

Step	Edge	RD[v] after join	Grows?
5	$5 \to 3$ **(back-edge)**	$\{(x,1),(x,5),(y,2),(y,4)\}$	✓ **grows!**
6	$3 \to 4$	$\{(x,1),(x,5),(y,2),(y,4)\}$	✓
7	$4 \to 5$	$\{(x,1),(x,5),(y,4)\}$	✓
8	$5 \to 3$	$\{(x,1),(x,5),(y,2),(y,4)\}$	× **no change — sto**

Key insight: Step 5

Back-edge $5 \to 3$ injects $(x,5)$ and $(y,4)$ into RD[3], forcing re-propagation through the loop body (steps 6–7). Step 8 finds no new information $\Rightarrow$ fixed point reached.

Worklist Trace — Discussion

- **Step 5** is the critical one: the back-edge $5 \to 3$ feeds new definitions $(x, 5)$ and $(y, 4)$ into label 3. Since label 3's set *grew*, its outgoing edges are re-added to the worklist.
- **Step 6** re-processes edge $3 \to 4$ because label 3 changed. Label 4 now sees all four definitions and its set also grows, triggering further propagation.
- This cascading effect terminates because sets can only grow and there are finitely many definitions ($|D| = 4$ here).
- The final result confirms that **both** definitions of x (labels 1 and 5) and both definitions of y (labels 2 and 4) are simultaneously reachable at the loop condition — reflecting the two possible execution scenarios (first iteration vs. subsequent iterations).

Bit Vector Frameworks

Definition

A **Bit Vector Framework** has:

- Lattice $L = \mathcal{P}(D)$ for a finite domain D
- Every transfer function of the form:
  $f(\ell) = (\ell \setminus \ell_k) \cup \ell_g$ (kill/gen form)

Key Theorem

Every Bit Vector Framework is **distributive**.

Proof:

$$\begin{aligned} f(X \cup Y) &= ((X \cup Y) \setminus \ell_k) \cup \ell_g = (X \setminus \ell_k) \cup (Y \setminus \ell_k) \cup \ell_g \\ &= \left[(X \setminus \ell_k) \cup \ell_g\right] \cup \left[(Y \setminus \ell_k) \cup \ell_g\right] = f(X) \cup f(Y) \end{aligned}$$

Bit Vector Frameworks — Discussion

- The kill/gen form is exactly the transfer function shape shared by Reaching Definitions, Live Variables, Available Expressions, and Very Busy Expressions — all four are bit vector frameworks.
- Each element of D maps to **one bit**. The entire lattice state for a node fits in a machine word when $|D| \leq 64$.
- Transfer functions become a handful of **bitwise instructions**: `state &= ~kill_mask` (kill), then `state |= gen_mask` (gen).
- This is why classical dataflow inside production compilers (GCC, LLVM) is extremely fast even on large functions: each propagation step is $O(|D|/64)$ in time and space.

Bit Vector Representation

Each element of D is one **bit**; the lattice element is a bitvector.

Reaching Definitions on our example

Definitions: $d_1 = (x, 1)$, $d_2 = (y, 2)$, $d_3 = (y, 4)$, $d_4 = (x, 5)$

$$\text{State at label 3 (entry)} : \underbrace{1}_{d_1}\ \underbrace{1}_{d_2}\ \underbrace{1}_{d_3}\ \underbrace{1}_{d_4}$$

All four definitions reach the loop condition.

- Transfer = `&~` (kill) then `|` (gen) on integers
- Extremely fast: one instruction per 64 definitions

MOP Solution

Meet Over all Paths (MOP)

For each program point u, combine the transfer functions along *every* path π from the entry:

$$\mathbf{MOP}[u] = \bigsqcup_{\pi:\text{entry} \rightsquigarrow u} f_{\ell_n} \circ \cdots \circ f_{\ell_1}(\iota)$$

Paths to label 4 in our example:

- $1 \to 2 \to 3 \to 4$ (first iteration)
- $1 \to 2 \to 3 \to 4 \to 5 \to 3 \to 4$ (second)
- ... infinitely many paths through the loop

Problem

Because loops produce infinitely many paths, **MOP is generally undecidable**.

MOP Solution — Discussion

- MOP is the **theoretically ideal** solution — it never merges information from paths that are actually impossible at runtime.
- For **loop-free** programs (DAGs), MOP is computable: there are finitely many paths, and we can enumerate them.
- For programs **with loops**, the infinite path enumeration makes MOP undecidable in general. This is why we use MFP (worklist) as a computable approximation.
- The gap between MOP and MFP exists only when the framework is **not distributive** (e.g., Constant Propagation). For distributive frameworks (gen/kill), MFP = MOP exactly.

MFP vs MOP — The Key Relationship

Theorem

For any monotone framework:

$$\mathbf{MOP}[\ell] \sqsubseteq \mathbf{MFP}[\ell] \qquad \forall\, \ell$$

i.e., **MFP safely over-approximates MOP**.

Why? At join points, MFP merges information from all predecessors *before* applying the transfer function, losing path-sensitive distinctions that MOP keeps.

Theorem

If the framework is **distributive**, *then* **MFP** $=$ **MOP**.

MFP vs MOP — Discussion

- "**Safe over-approximation**" means MFP may claim *fewer* facts than MOP (e.g., MFP says $y = \top$; MOP says $y = 1$), but it *never* makes a false claim. Any conclusion drawn from MFP is valid for all program executions.
- The precision loss comes from **merging at join points**: MFP joins all incoming paths before applying f, while MOP applies f along each path separately then joins.
- For **safety-critical** analysis (e.g., null pointer detection), MFP's conservatism is a feature: it never misses a bug, though it may report false positives.
- Distributive frameworks close the gap entirely: MFP = MOP, so we get ideal precision at polynomial cost.

Intuition: Why Distributivity Implies MFP = MOP

Distributivity: $f(\ell_1 \sqcup \ell_2) = f(\ell_1) \sqcup f(\ell_2)$

MFP (merge then apply):

$$f\left(\underbrace{\ell_1 \sqcup \ell_2}_{\text{join at merge}}\right)$$

MOP (apply then merge):

$$\underbrace{f(\ell_1) \sqcup f(\ell_2)}_{\text{join after propagation}}$$

When distributive, both sides are *equal* — order does not matter.

Consequence for Constant Propagation

Not distributive $\Rightarrow$ MFP $\sqsupset$ MOP. The $x * x$ example: MFP reports $y = \top$; MOP deduces $y = 1$.

Distributivity and MFP = MOP — Discussion

- Distributivity is the **formal condition** that captures "no information is lost at merges."
- For **gen/kill analyses**, it holds because set union distributes over set difference: $(X \cup Y) \setminus k = (X \setminus k) \cup (Y \setminus k)$.
- For **Constant Propagation**, it fails because arithmetic on $\top$ discards algebraic structure (e.g., $x^2 \geq 0$ is true for all x, but $\top^2 = \top$ in the lattice).
- This explains the design choice in practice: **use distributive analyses when precision matters**; accept MFP $\sqsupset$ MOP for richer analyses like Constant Propagation, knowing results are still sound.

Summary

Analysis	**Dir.**	**Distr.?**	**MFP=MOP?**
Reaching Definitions	Fwd	Yes	Yes
Available Expressions	Fwd	Yes	Yes
Live Variables	Bwd	Yes	Yes
Very Busy Expressions	Bwd	Yes	Yes
Constant Propagation	Fwd	No	No

Framework Hierarchy

$$\text{Bit Vector} \subset \text{Distributive} \subset \text{Monotone}$$

Take-away

Use the Worklist algorithm to compute MFP efficiently. For distributive frameworks, MFP = MOP: maximum precision at polynomial cost.

Summary — Discussion

- The **hierarchy** tells us:
  - Bit-vector analyses: fastest (bitwise ops), always distributive, MFP = MOP.
  - Distributive analyses: equally precise (MFP = MOP), may not be bitvector.
  - Monotone analyses: safe but may over-approximate (MFP $\sqsupset$ MOP).
- When designing or choosing an analysis, always ask: **is it distributive?** If yes, the worklist gives the ideal result. If no, consider whether the added expressiveness is worth the precision loss.
- All results from MFP are **sound** regardless of distributivity — the difference is only in *precision*, not correctness.